# OptionQ: A Regime-Aware, Multi-Layer Options Hedge Engine with LLM-Augmented Explanation

Dipen Prajapati

**Abstract**

Managing downside risk in an equity portfolio requires choosing the right instrument, sizing it correctly, and understanding why it fits. Most retail tools handle one of those steps, not all three. OptionQ is an end-to-end hedge recommendation engine that takes a portfolio as input and returns ranked, priced, and sized hedge candidates, each paired with a plain-English rationale generated by a large language model. The system uses a dual Gaussian Mixture Model (GMM) regime detector to classify current market conditions, then filters and scores candidates across cost efficiency, delta coverage, liquidity, and regime fit. I built this as a full-stack project with a FastAPI backend running a 12-layer analysis pipeline and a Next.js frontend for real-time interaction.

---

## 1 Introduction

Options are one of the most flexible tools for managing portfolio risk. But using them well requires getting three things right at the same time: picking the correct instrument, sizing it to your actual exposure, and choosing an expiry that matches your hedge horizon. Getting any one of these wrong can leave a portfolio partially unhedged or overpaying for protection.

Most retail platforms give you a pricing calculator or a strategy screener. They don't connect those pieces into one coherent recommendation. I built OptionQ to close that gap. The goal is a system where you describe your holdings, set a cost budget, and get back specific hedge candidates, complete with contract counts, cost estimates, Greeks, and a clear explanation of why each one fits your position.

The system is regime-aware, which matters because the right hedge changes depending on market conditions. A deep out-of-the-money put is cheap in calm markets but gets expensive fast when volatility spikes. OptionQ detects the current regime first, then uses that context to filter and rank candidates, so the recommendations reflect what the market looks like today rather than what it looked like on average.

## 2 System Architecture

OptionQ is a monorepo with a Python backend and a TypeScript frontend. The backend is a FastAPI application that exposes a `/portfolio/analyze` endpoint. Each request triggers a 12-layer pipeline, where every layer enriches or filters the candidate set before passing it to the next stage. Table 1 summarizes each layer.

The pipeline treats stock and options positions differently. For a stock holding, it hedges based on share count and current spot price. For an existing options position, it computes the effective

Table 1: The 12-Layer Hedge Pipeline

| Layer | Name | Purpose |
|---|---|---|
| 1 | Input Parser | Validate and normalize portfolio input |
| 2 | Market Context | Regime detection via dual GMM + GARCH |
| 3 | Risk Analysis | Beta, VaR, and hedge ratios per holding |
| 4 | Instrument Selection | Rule-based hedge candidate generation |
| 5 | Prelim Explainer | Optional early LLM explanation pass |
| 6 | Pricing Engine | BSM / Black-76 / GK / Monte Carlo pricing |
| 7 | Position Sizer | Delta-target sizing with budget gate |
| 8 | Greeks Engine | Portfolio-level Greeks computation |
| 9 | Simulation | Monte Carlo and stress scenario analysis |
| 10 | Evaluator | Composite 0–100 candidate scoring |
| 11 | Final Explainer | LLM rationale with full quant context |
| 12 | Output Formatter | Sort, rank, trim, and timestamp output |

delta-adjusted equity exposure first, then routes that through the same pipeline. This means a long call position gets hedged based on its net directional exposure, not its notional value.

The frontend is a Next.js application that calls the backend through a typed API client. It renders hedge candidates as expandable cards, each showing the instrument, expiry, contract count, cost, coverage percentage, Greeks, and the LLM explanation.

# 3 Core Methodology

## 3.1 Regime Detection

Before generating any candidates, the pipeline classifies the current market regime. I use two Gaussian Mixture Models (GMM) [4, 5] running in parallel whose outputs are blended into a single regime signal.

The *main model* trains on 10 years of daily macro data with 4 GMM components and retrains monthly. It anchors the regime signal in a full market cycle so the thresholds reflect bull, bear, and recovery periods. The *fast model* uses a 6-month rolling window with 3 components and retrains daily, which lets it pick up sudden regime shifts like a VIX spike that the slow model is too coarse to catch immediately.

Both models train on 10 features: VIX level, 1-day and 5-day VIX change, 20-day realized volatility, SPY 5-day return, VIX term structure slope, TLT 5-day return, HYG 5-day return, IV/RV ratio, and 20-day SPY-TLT correlation. Each model produces two independent anomaly signals: a VIX percentile score (how extreme is today's VIX relative to the training window) and a GMM uncertainty score (1 minus the maximum component probability, which is high when the current observation doesn't fit any known regime well). These are blended into a single per-model anomaly score using a configurable weight. The main model weights the VIX signal at 0.65 since its 10-year training data gives GMM reliable cluster structure. The fast model uses an equal 0.50 blend because its 6-month window produces tighter clusters that can overfit to recent calm.

The two model scores are then fused as:

$$s_{\text{combined}} = 0.65 \cdot s_{\text{main}} + 0.35 \cdot s_{\text{fast}} \tag{1}$$

The regime is flagged as an anomaly if $s_{\text{combined}} \geq 0.60$, or if either model individually exceeds

0.85 as a hard override. The final label (low_vol, mid_vol, high_vol, or anomaly) is determined by snapping a confidence-weighted ordinal average of both models' labels back to the nearest class. This means a high-confidence fast model can shift the combined label even when the slow model disagrees, but only proportionally to its confidence. The regime label gates candidate scoring: put options score higher in high-volatility and anomaly regimes, while covered strategies score higher in low-volatility regimes.

### 3.2 Candidate Generation and Filtering

Layer 4 generates hedge candidates for each holding. For equity positions, the options selector checks two candidate pools: direct options on the underlying ticker and cross-hedge options on correlated instruments like sector ETFs or broad index proxies such as SPY and QQQ.

Every candidate passes through a liquidity gate before being included. The gate checks open interest (minimum 100 contracts) and the bid-ask spread as a percentage of mid price (maximum 5%). Illiquid candidates are dropped rather than penalized in scoring, which keeps the final output free of strikes that would be hard to execute.

Implied volatility for each candidate is fetched from the live options chain using linear interpolation across the volatility smile. Given two bracketing strikes $K_{\text{lo}}$ and $K_{\text{hi}}$ with implied volatilities $\sigma_{\text{lo}}$ and $\sigma_{\text{hi}}$, the interpolated IV at target strike $K$ is:

$$\sigma(K) = \sigma_{\text{lo}} + \frac{K - K_{\text{lo}}}{K_{\text{hi}} - K_{\text{lo}}} \cdot (\sigma_{\text{hi}} - \sigma_{\text{lo}}) \tag{2}$$

This correctly accounts for the volatility skew, where out-of-the-money puts carry higher implied volatility than at-the-money options [7].

### 3.3 Pricing and Sizing

Equity options are priced using the Black-Scholes-Merton model [1]. For a put with underlying price $S$, strike $K$, time to expiry $T$, risk-free rate $r$, dividend yield $q$, and implied volatility $\sigma$:

$$P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1) \tag{3}$$

where $d_1 = \frac{\ln(S/K)+(r-q+\sigma^2/2)T}{\sigma\sqrt{T}}$ and $d_2 = d_1 - \sigma\sqrt{T}$. FX options use the Garman-Kohlhagen model [3] and options on futures use the Black-76 model [2].

**Per-strike implied volatility.** The volatility input $\sigma$ is resolved per candidate strike rather than using a single at-the-money (ATM) value. In Layer 6, the pricing engine calls `get_strike_iv()` for each candidate, which looks up the actual implied volatility at that specific strike from the live options chain using the linear interpolation formula in Equation 2. If the exact strike is not present, the function interpolates between the two nearest bracketing strikes or falls back to the nearest available strike within 15%. The ATM IV is used only as a last resort when no chain data is available. For multi-leg strategies (spreads, collars), each leg is resolved independently with its own per-strike IV. This ensures that out-of-the-money puts, which carry higher implied volatility due to the skew, are priced correctly rather than being systematically underpriced against an ATM baseline [7].

### 3.4 Market Premium vs. Model Price

Every candidate in the output carries two cost figures. The *model price* is the BSM-computed theoretical premium and is used for Greeks and sizing calculations. The *market mid price* is the actual transaction cost a trader would pay, computed directly from the live options chain as:

$$P_{\text{mid}} = \frac{\text{bid} + \text{ask}}{2} \tag{4}$$

For multi-leg strategies the net market mid is $P_{\text{mid}} = P_{\text{mid,long}} - P_{\text{mid,short}}$, floored at zero. The total market cost displayed to the user is:

$$C_{\text{market}} = P_{\text{mid}} \times 100 \times n \tag{5}$$

This distinction matters in practice. BSM assumes continuous hedging and no transaction costs; the options market does not. Displaying the market mid alongside the model price makes the bid-ask spread impact visible at a glance and gives the user a realistic cost figure before placing an order.

Sizing runs in two stages. The first stage targets a delta that matches the desired protection level. Given a target delta $\Delta_{\text{target}}$ and a per-contract delta $\Delta_c$, the initial contract count is:

$$n_{\text{target}} = \left\lfloor \frac{\Delta_{\text{target}} \cdot N_{\text{holding}}}{|\Delta_c| \cdot 100 \cdot S} \right\rfloor \tag{6}$$

The second stage applies a budget gate. Given a maximum cost as a percentage of portfolio notional $c_{\max}$ and a per-contract premium $P_c$:

$$n_{\text{budget}} = \left\lfloor \frac{N_{\text{portfolio}} \cdot c_{\max}}{P_c \cdot 100} \right\rfloor \tag{7}$$

The final contract count is $n = \min(n_{\text{target}}, n_{\text{budget}})$. The coverage percentage shown on the UI is:

$$\text{Coverage\%} = \frac{n \cdot |\Delta_c| \cdot 100 \cdot S}{N_{\text{holding}}} \times 100 \tag{8}$$

This two-stage approach makes the trade-off visible. If the budget gate is binding, the user can see that coverage is below 100% and decide whether to raise the cost budget or accept partial protection.

### 3.5 Greeks and Per-Position Display

Layer 8 computes option Greeks for each candidate scaled to the full position size. The portfolio-level delta stored on each candidate is:

$$\Delta_{\text{pos}} = \Delta_c \times n \times 100 \tag{9}$$

where $\Delta_c$ is the per-contract Black-Scholes delta and $n$ is the number of contracts. Gamma and vega are scaled similarly. The frontend displays per-contract Greeks by reversing this scaling: the UI divides $\Delta_{\text{pos}}$ by $n \times 100$ so the user sees the delta sensitivity per contract rather than the aggregate

position. This is shown independently for each holding's top candidate, not as a combined portfolio total.

### 3.6 Scoring and Ranking

Layer 10 scores each candidate on a 0–100 composite scale across four dimensions. Cost efficiency measures the premium paid relative to the protection value delivered. Delta coverage rewards candidates that cover a higher percentage of the holding's notional. Liquidity rewards tighter bid-ask spreads and higher open interest. Regime fit adjusts scores based on the current regime label, where put-heavy candidates score higher in stressed and crisis regimes. The top three candidates per holding are returned, sorted by composite score.

## 4 LLM Explainer Layer

One of the reasons options hedging is underused is that quant model output is hard to act on. A delta of $-0.35$ doesn't tell a retail investor much by itself. Layer 11 addresses this by generating a plain-English rationale for each candidate using a large language model.

The explainer is pluggable and supports Anthropic's Claude API, a locally-hosted Ollama instance (such as Llama 3), and HuggingFace models. A shared prompt builder constructs a structured context object for each candidate that includes the underlying position, current regime, candidate Greeks, cost, coverage, and expiry. The LLM produces a two to three sentence explanation of why that specific hedge fits the user's position given current market conditions.

This layer is non-blocking. If the LLM call fails or times out, the pipeline continues and returns candidates without explanations rather than surfacing an error to the user.

## 5 Implementation

The backend is a FastAPI application running on Python 3.13 with Uvicorn. An APScheduler instance manages model retraining jobs in the background: the fast regime model retrains daily and the main model retrains on the first of every month. Market data comes from yfinance [6], FRED for risk-free rate data, and the VIX term structure. Responses are cached in SQLite to avoid redundant API calls within a session window.

The frontend is a Next.js 16 application using Tailwind CSS. The main page lets users add stock and options positions, configure hedge constraints (cost budget, protection level, hedge horizon), and view ranked candidates in real time.

**Production-ready:** The full 12-layer pipeline, options candidate generation, BSM/Black-76/GK/MC pricing with per-strike IV surface interpolation, market mid price display alongside model price, delta-target sizing, per-position Greeks computation and display, Monte Carlo simulation, composite scoring, LLM explanation, dual-model regime detection, and the frontend UI.

## 6 Limitations and Future Work

The current system generates hedge candidates but does not execute them. The Alpaca execution layer is planned but not yet wired, so the workflow ends at a recommendation the user must act on manually.

The instrument universe is also limited to equity options. The futures, forwards, and swaps selectors exist as routing stubs that return empty candidate lists until the pricing and sizing logic for each is implemented. Expanding these would make the system useful for multi-asset portfolios beyond

equity books.

The system already filters candidates by earnings proximity: expiries that cross an upcoming earnings date are penalized in the scoring step to reduce IV crush risk. What is not yet integrated is a richer sentiment layer that uses FinBERT on earnings transcripts to adjust the regime or scoring signal based on company-specific news rather than only macro volatility indicators.

A backtesting harness is also on the roadmap. This would let users see how a hedge strategy would have performed over a historical period given a specific regime sequence, which is important for validating the regime-gated scoring logic against real drawdown events.

## References


[1] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.

[2] F. Black, "The pricing of commodity contracts," *Journal of Financial Economics*, vol. 3, no. 1–2, pp. 167–179, 1976.

[3] M. B. Garman and S. W. Kohlhagen, "Foreign currency option values," *Journal of International Money and Finance*, vol. 2, no. 3, pp. 231–237, 1983.

[4] D. A. Reynolds, "Gaussian mixture models," *Encyclopedia of Biometrics*, pp. 741–749, 2009.

[5] A. Botte and D. Bao, "A machine learning approach to regime modeling," *Two Sigma Whitepaper*, 2021.

[6] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Pearson Education, 2017.

[7] J. Gatheral, *The Volatility Surface: A Practitioner's Guide*. Wiley Finance, 2006.